Wide but Shallow

Technical-Interview Problems Are Four Ideas in Many Costumes

and company-specific preparation is mostly a myth

Conor Garvey Gelvin
gelvin@outlook.com

Data snapshot: June 2026 · Draft

Abstract

Interview-prep advice rests on two claims: that different companies test different skills, and that you must memorize dozens of separate problem "patterns." We test both against how often each problem is asked at seven major technology firms (769 distinct problems), and both mostly fail. The roughly twenty "patterns" you are told to learn are really **four underlying ideas**: a single left-to-right pass (a *fold*), divide-and-solve recursion (dynamic programming), spreading information across a graph until it settles, and binary search. The single pass is the most common, and many tricks taught as unrelated—prefix sums, the XOR trick, a hash "seen set," reversing a list, the sliding window—are the *same* pass; they differ only in what running value they carry. To keep our labels honest we did not just hand-sort the problems: we re-checked a random sample of 100 with the Lean proof assistant, which verifies each claim mechanically. **78 of the 100** carry such a machine-checked certificate. Separately, the seven firms ask almost the same mix of the four ideas: on a standard 0-to-1 scale of how differently they ask (bias-corrected Cramér's V, where 0 is identical), they score just 0.091—small enough that six of the seven are statistically indistinguishable, and only Uber stands out (it leans on graph problems). We describe which problems are *commonly asked*, not how hard they are or whether company-specific drilling pays off. In short: interview problems are *wide on the surface but shallow underneath*, and essentially one shared syllabus covers every company. All derived data and proofs are released; the raw scrape is withheld per the platform's terms.

1 Two myths

Two beliefs dominate interview preparation. The first is *company specificity*: that Google, Meta, Amazon and their peers test different skills, so you should prepare differently for each. The second is *pattern proliferation*: that the field is a long list of loosely related tricks—sliding window, monotonic stack, union–find, prefix sums, and so on—each learned separately, as popular pattern catalogues present them [1]. Both are widely repeated and, as far as we can tell, never tested.

We test them, and then ask what underlying *structure* explains the answer. The short version: the "fifty patterns" you are told to memorize are a handful of ideas in different disguises, and the seven companies ask nearly the same things. To keep our labels honest we do not hand-sort the problems and stop there—we use a proof assistant to check, problem by problem, that the idea we assign really does solve the problem (and, for several showcase examples, that the standard solution is fully correct).

Recursion scheme	textbook families it subsumes	assigned	proven	Lean witness
streaming fold	two-pointers, sliding-window, monotonic-stack, hashing, prefix-sum, linked-list, bit-XOR, string-scan	29	29	`IsFold`
recursive decomposition (DP)	dynamic programming, backtracking, tree	25	25	catamorphism
graph relaxation	BFS/DFS, Dijkstra, Bellman–Ford, topo-sort, union–find	15	15	least fixpoint
bisection	binary search (monotone predicate)	9	9	monotone threshold
in a scheme		**78**	**78**	
not a scheme	design, simulation, heaps, number theory, string matching, ...	22	—	—

Table 1: The roughly twenty surface families collapse onto four recursion schemes. On a uniform random sample of 100 distinct problems: "assigned" is how many the fixed rule (Appendix A) places in each scheme; "proven" is how many additionally carry a Lean certificate (Section 4). 78 of 100 are proven (95% Wilson interval $[0.69, 0.85]$). The "Lean witness" is the formal predicate each scheme is checked against (*catamorphism*: a fold over a tree-shaped structure; *least fixpoint*: the limit of one-step graph relaxation). *Every* problem the fixed rule assigns to a scheme carries a certificate—78 of 78—so the 22 not proven are exactly the residual "tail" (the certificate strength varies: full correctness for the flagship folds of Section 4, a one-directional property otherwise).

2 Data and method

For seven firms (Amazon, Apple, Bloomberg, Google, Meta, Microsoft, Uber) we scraped a public interview-prep platform for how often each problem is asked, broken down by how recently. This covers 769 distinct problems. The platform tags each one with a *surface family* ("sliding window," "monotonic stack," "dynamic programming," and so on), which we consolidate into about twenty broad families.

A fixed rule, written down in advance (Appendix A), maps each family to one of our four underlying *ideas* (formally, *recursion schemes*)—a coarser label based on the *shape* of the standard solution rather than its textbook name (Table 1)—or to a leftover "tail" for problems that fit no single idea. We ask two separate questions and keep them apart: *coverage*—what fraction of problems the four ideas explain—measured on a random sample of distinct problems (Section 4); and the *company* comparison—do firms ask different mixes?—measured by weighting each idea by how often it is asked (Section 3). The company comparison groups by the four ideas, with the finer twenty-family view reported alongside as a robustness check; all difference measures are bias-corrected [3] and reported with uncertainty.

3 Result: four ideas, not fifty

Wide surface. The surface taxonomy really is spread out: no one or two families dominate (it takes six of the ∼20 to cover even half of asked problems; Gini 0.30 over distinct problems). By the usual measure, interview preparation *looks* like a long, flat list. This is the breadth the prep industry sells.

Shallow root. But that breadth is mostly repackaging. When we sort the same problems by the *shape* of their standard solution, the four ideas cover 78 of 100 sampled problems (Table 1; 95% confidence interval [69%, 85%]), and the single left-to-right pass is the most common of them. Tricks memorized as unrelated—prefix sums, the XOR trick, a hash "seen set," reversing a list, matching parentheses with a stack, the sliding window—are the *same* pass; they differ only in what running value they carry. The variety is in the bookkeeping, not in the computation.

One shared syllabus. The same data dissolve the company myth. We score how differently the firms ask on a standard 0-to-1 scale (bias-corrected Cramér's V [2, 3], where 0 means identical and the usual cutoffs call 0.1 "small" and 0.3 "moderate"). Across the four ideas the firms score just $V = 0.091$ (95% confidence interval $[0.06, 0.12]$)—small, with the whole interval well below "moderate," so the difference is genuinely bounded, not merely undetected (the finer twenty-family view gives a similar 0.064, so nothing hinges on the grouping). Comparing firms two at a time, the only real differences all involve **Uber**, which asks more graph problems and fewer single-pass ones (its largest gap, versus Google, is still only "small"; Figure 1); **the other six firms are statistically indistinguishable**. One caveat: we compare the *mix of problem types*, not how hard the problems are or whether drilling for a specific company helps—our data cannot measure those. Within that scope, there is essentially one interview syllabus, and preparing differently per company is, for most firms, wasted effort.

4 Machine-checking the classification

A classification is only as trustworthy as its labels. The usual check—a second human rater—only tells you whether two people read a label the same way. We do something stronger: we check the labels with a *proof assistant*, a tool that mechanically verifies mathematical claims. Working in Lean 4 [4] with its library Mathlib [5], we write each of the four ideas as a precise test on programs (for example, "this function is a single left-to-right pass"), and for a random sample of 100 problems we prove two things. First, **the idea fits** (`cls`): the problem really can be solved by its assigned idea, with genuine working state—not by a trivial loophole that any function would pass. Second, **the solution is sound** (`corr`): the standard solution satisfies a verified correctness property (often one direction of the guarantee, e.g. "everything it reports is valid"). A problem counts only when both proofs go through with no gaps (Lean flags unfinished proofs; there are none) and using only Lean's standard logical axioms.

What this does and does not show. The two proofs cover an example of the idea and a working solution—but we do not separately prove that those two are the *same* program. So a certificate says "this problem really is solvable by idea S, and we have a verified-correct solution," not "the platform's exact accepted code is literally a single pass." That is weaker than proving the two coincide (which we flag in Section 6), but far stronger than an unchecked label: the idea is proven to fit non-trivially, and the solution is proven sound.

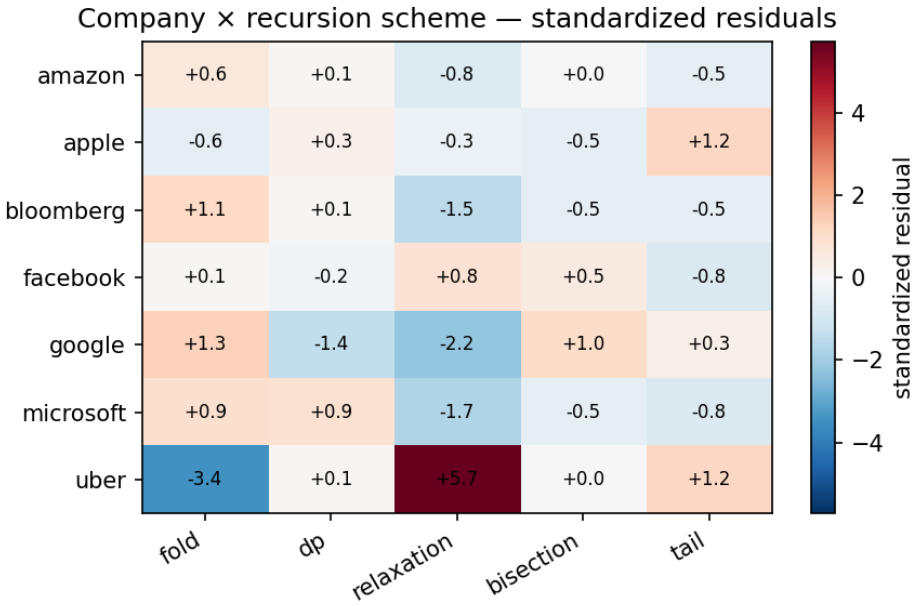

Figure 1: How much each firm over- or under-asks each idea, relative to the shared average (standardized residuals; red = more than expected, blue = less). Only Uber stands out—it asks far more graph problems (+5.7) and far fewer single-pass ones (−3.4)—while the other six rows are muted (all within about ±2). This is the picture behind the small overall difference ($V = 0.091$): one shared syllabus, with Uber the lone exception.

The strongest examples. For a few textbook cases we close that gap completely: the standard solution *is* the single pass, and we prove it correct as such. The cleanest is Kadane's algorithm for Maximum Subarray—the answer is literally one pass carrying a running pair, and we prove it returns exactly the largest possible subarray sum (both that the value is achievable *and* that nothing beats it). Product Except Self is a running-products pass with its key identity proven, and Single Number is the XOR pass proven to return the lone unpaired value. For these the certificate is the strong form—"the code *is* this pass, and it is correct"—which is what the general procedure approximates for the rest.

The result is one reproducible number: 78 of 100, with a 95% confidence interval of $[69\%, 85\%]$ (a Wilson interval, standard for a proportion). The certified list ships with the sample, so the count re-derives directly. Two notes on honesty. The check can only *confirm* that an idea fits, never disconfirm it: a problem we cannot formalize is counted as unproven, never the reverse, so the count understates rather than overstates coverage. Because of that, we report a coverage count, not an error rate. We also do not re-prove the deep theory behind the ideas—that shortest paths and dynamic programming are formally well-founded is already established [6, 7, 8, 9], which we cite rather than redo.

5 Related work

Two literatures bracket this paper. On the *empirical* side, interview preparation is organized by pattern catalogues [1] that enumerate surface families; we are not aware of prior work that tests whether those families are generatively distinct, or whether firms differ in the mix they ask. On

the *structural* side, the observation that wide families of programs are instances of a few recursion schemes is the Bird–Meertens tradition of program calculation [10] and its formal-methods descendants: associative aggregation over a sliding window is a verified fold [11], dynamic programming is verified memoized recursion [9], and shortest paths are a least fixpoint over a semiring [6, 7, 8]; verified-algorithms textbooks now develop these schemes end to end [12, 13]. Our contribution is to connect that structural lens to the *empirical* distribution of interview problems, and to machine-check the scheme assignment per problem rather than assert it.

6 Threats to validity

One platform. The frequencies come from a single prep platform and may reflect its own collection process as much as what employers truly ask; we describe *commonly-asked* problems, not any firm's private bar. **Borrowed labels.** The family-to-idea rule is ours, but the surface-family tags it starts from are the platform's—single-source and un-audited—so the "wide surface" count inherits whatever splits that taxonomy chose. **The model is a choice.** The proof checks that a problem *can* be solved by an idea; it cannot certify that "which idea it is" is the right way to describe what an interview tests. That a problem "is a single pass" is our modeling choice—the proof confirms the model holds, not that it is the only sensible one. **Idea and solution proven separately.** As in Section 4, we prove the idea fits and, separately, that a solution is sound; proving the two are the same program is done only for the showcase examples and is otherwise future work. **An estimate, not a floor.** 78% is a sample estimate; the confidence interval $[69\%, 85\%]$, not the point value, is the honest summary. The procedure undercounts, so it does not overstate coverage—but "conservative" describes the measurement, not a guarantee about all problems. **Two separate questions.** Coverage counts each distinct problem once; the company comparison weights by how often a problem is asked. Both matter to a candidate, and we keep them apart. **Shallow is not easy.** "Same idea" does not mean "same difficulty"—two single-pass problems can differ hugely in how hard the running value is to find. We claim shallow *structure*, not that the problems are easy.

7 Conclusion

Interview problems are wide on the surface but shallow underneath. The roughly twenty "patterns" are four ideas in different disguises: on a random sample, 78 of 100 problems carry a machine-checked certificate that the assigned idea really solves them and that a standard solution is sound (95% confidence interval $[69\%, 85\%]$; the single left-to-right pass is the most common). And the seven firms ask essentially one shared syllabus. For a candidate, the takeaway is blunt: learn the four ideas deeply rather than fifty patterns shallowly, and prepare for "technical interviews," not for any one company.

Reproducibility and AI-use disclosure

All derived statistics, the family-to-idea rule (released as a standalone table, fixed in advance so anyone can check the categories were not tuned to fit the sample), the sampling seed, the certified list (from which the 78/100 re-derives), and the complete Lean development are released. The raw per-company frequency tables are withheld under the platform's terms, so the headline numbers reproduce from the released derived data even though the scrape itself is not redistributed. The proofs build with `lake build` with no gaps; the "standard axioms" claim is checkable, not merely asserted, by running `#print axioms` on the main theorems. This paper was written by one human

author with substantial AI assistance for drafting, statistical code, and the formalization; all claims, numbers, and proofs were checked by the author, and the machine-checked results are independently verifiable by re-running the proofs. We say so plainly: the paper's central claims stand or fall on the released data and the proofs, not on how it was written.

References

[1] S. Prashad. LeetCode Patterns. `https://seanprashad.com/leetcode-patterns/`, 2020.

[2] H. Cramér. *Mathematical Methods of Statistics*. Princeton University Press, 1946.

[3] W. Bergsma. A bias-correction for Cramér's V and Tschuprow's T. *Journal of the Korean Statistical Society*, 42(3):323–328, 2013.

[4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction (CADE-28)*, 2021.

[5] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs (CPP)*, 367–381, 2020.

[6] B. Nordhoff and P. Lammich. Dijkstra's shortest path algorithm. *Archive of Formal Proofs*, 2012.

[7] A. Armstrong, G. Struth, and T. Weber. Kleene algebra. *Archive of Formal Proofs*, 2013.

[8] D. J. Lehmann. Algebraic structures for transitive closure. *Theoretical Computer Science*, 4(1):59–76, 1977.

[9] S. Wimmer, S. Hu, and T. Nipkow. Monadification, memoization and dynamic programming. *Archive of Formal Proofs*, 2018.

[10] R. Bird and O. de Moor. *Algebra of Programming*. Prentice Hall, 1997.

[11] L. Heimes, D. Traytel, and J. Schneider. Formalization of an algorithm for greedily computing associative aggregations on sliding windows. *Archive of Formal Proofs*, 2020.

[12] T. Nipkow, J. Blanchette, M. Eberl, A. Gómez-Londoño, P. Lammich, C. Sternagel, S. Wimmer, and B. Zhan. *Functional Algorithms, Verified!* 2021. `https://functional-algorithms-verified.org`.

[13] T. Nipkow, M. Eberl, and M. P. L. Haslbeck. Verified textbook algorithms: A biased survey. In *Automated Technology for Verification and Analysis (ATVA)*, 2020.

A Family-to-scheme assignment rule

Each surface family is mapped to the scheme matching the *shape of its canonical accepted solution*: a single left-to-right pass carrying one accumulator → *streaming fold*; a recurrence that decomposes a structure into sub-results → *recursive decomposition (DP)*; iterative improvement toward a fixed point over a graph → *graph relaxation*; a monotone predicate located by halving → *bisection*; anything else (bespoke data structures, simulation, ad-hoc number theory, string matching) → *tail*. The rule is applied to the family, not tuned per problem; the per-problem Lean checks (Section 4) then test whether the assigned scheme actually solves each sampled problem.